```
object Wallet {

  def apply(): Behavior[Int] =
    Behaviors.receive { (context, message) =>
      context.log.info(s"received '$message' dollar(s)")
      Behaviors.same                                          ❶
    }
}
```

❶ The next message will be processed by the same behavior.

Note that the `Behaviors` are nested like Russian dolls. Here the outer `def apply(): Behavior[Int]` is composed with a `Behaviors.receive` that contains a `Behaviors.same`. You'll see this pattern over and over.

2.2.5 Terminating the system

You should terminate the `ActorSystem` gracefully so the system can shut down all running modules that hold, for example, HTTP or database connections. To do so, you terminate the guardian (the `ActorSystem`) with `.terminate()`.

Because this example will run inside the sbt shell, you can add a hook to the shell to trigger the termination, such as listening for an Enter keystroke. You can do this with `scala.io.StdIn.readLine()`, which is triggered by pressing Enter:

```
object WalletApp extends App {
  ...
  println("Press ENTER to terminate")
  scala.io.StdIn.readLine()
  guardian.terminate()
}
```